Question 15

Q: Given an application like OCR (Optical Character Recognition), explain how sampling, quantization, and image acquisition collectively affect performance.

Theoretical Concept: OCR Performance Factors

Sampling: Must be high enough to capture the curves and gaps of fonts. Too low = characters merge together (e.g., 'c' and 'l' become 'd').

Quantization: Document scanners usually binarize images (1-bit quantization: black or white). If done poorly (bad thresholding), text is lost.

Acquisition: Blur or uneven page illumination will ruin OCR accuracy.

💻 OpenCV Python Solution

Copy

```
import cv2  # Import OpenCV for image processing algorithms
import numpy as np                                          # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt                             # Import Matplotlib to visualize the output on screen

# Step 1: Create a white page document with dark text
page = np.full((100, 200), 255, dtype=np.uint8)  # Create a solid gray image array filled with a value
cv2.putText(page, 'OCR TEST', (20, 60), cv2.FONT_HERSHEY_SIMPLEX, 1.2, 0, 3)  # Render text characters onto the ima
```

```
# Step 2: Add a synthetic gradient shadow simulating bad illumination
gradient = np.tile(np.linspace(1.0, 0.1, 200), (100, 1))  # Repeat an array to build a larger image or gradient
shadow_page = (page * gradient).astype(np.uint8)

# Step 3: Use Adaptive Thresholding (1-bit Quantization optimization)
# This calculates thresholds locally, completely erasing the shadow gradient!
binary = cv2.adaptiveThreshold(shadow_page, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 21, 10)  # Appl

# Step 4: Display the OCR prep step
plt.figure(figsize=(8,4))  # Create a new figure window for display
plt.subplot(121), plt.imshow(shadow_page, cmap='gray'), plt.title('Poor Illumination')  # Place this image in a sub
plt.subplot(122), plt.imshow(binary, cmap='gray'), plt.title('Adaptive Thresholded')  # Place this image in a subpl
plt.show()  # Show the final plot window to the user
```

🎯 Expected Output

The plot shows a heavily shadowed document on the left where text vanishes into the darkness. On the right, adaptive thresholding flawlessly restores it to stark black-and-white text for OCR engines.

🚀 How to Execute & Submit

1. Google Colab Execution:

- Open Google Colab and create a New Notebook.
- Click the 📋 **Copy** button on the code block above.

- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File** > **Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.